E-Commerce Order Management System (E-OMS)

Behavioural Design Patterns — Technical Report (Deliverable #3)

Project: NexOrder — E-Commerce Order Management System (E-OMS)

---

1. Introduction

This report documents the **behavioural design pattern cases** identified and implemented in the E-OMS project.
For each required pattern/case, the report includes:

- **Problem statement** from the project context
- **Pattern-based solution overview**
- **UML class diagram**
- **Justification** for selecting the pattern and how it solves the problem
- **Implementation status** (where it exists in the Java project)

Required behavioural patterns covered:

- **Template Method** (2 cases)
- **Mediator** (2 cases)
- **Strategy** (2 cases)
- **Observer + Singleton** (1 combined case)
- **Iterator** (1 case)

---

2. Template Method Pattern

2.1 Case TM1 — Order Processing Workflow (Physical vs Digital)

Problem

Order processing follows a consistent high-level workflow (validate → payment → shipping/finalize), but **steps differ** depending on the product type:

- **Physical orders** require shipping and shipment persistence.
- **Digital orders** skip shipping but still require payment and finalization.

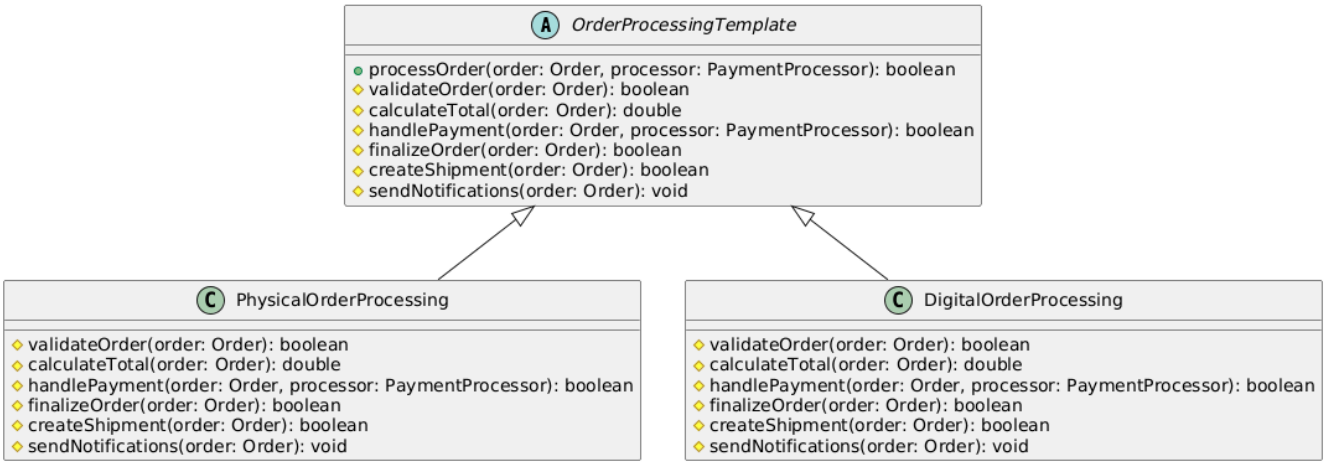
If implemented using large conditional blocks inside one method, the order-processing logic becomes:

- Hard to maintain and test
- Hard to extend (new order types require modifying existing code)
- More prone to duplicated logic

Template Method Solution

Define a common **algorithm skeleton** in a template class, while letting subclasses override specific steps.
The mediator selects the appropriate implementation (physical vs digital) at runtime, while preserving a stable workflow contract.

UML



Justification

Template Method is appropriate because:

- The workflow structure is stable, while individual steps vary by order type.
- It enforces consistent processing across order types.
- It improves extensibility by adding new order processing variants via new subclasses.

Implementation Status

Implemented in the project under the **template method** module and coordinated by the mediator:

- `OrderProcessingTemplate`
- `PhysicalOrderProcessing`
- `DigitalOrderProcessing`

2.2 Case TM2 — Report Export Workflow (Sales vs Inventory)

Problem

Admin report generation follows a consistent workflow:

- validate parameters → collect data → format/export

But the **data content differs** depending on report type:

- **Sales report** emphasizes product pricing
- **Inventory report** emphasizes product stock

Without structuring, report generation risks:

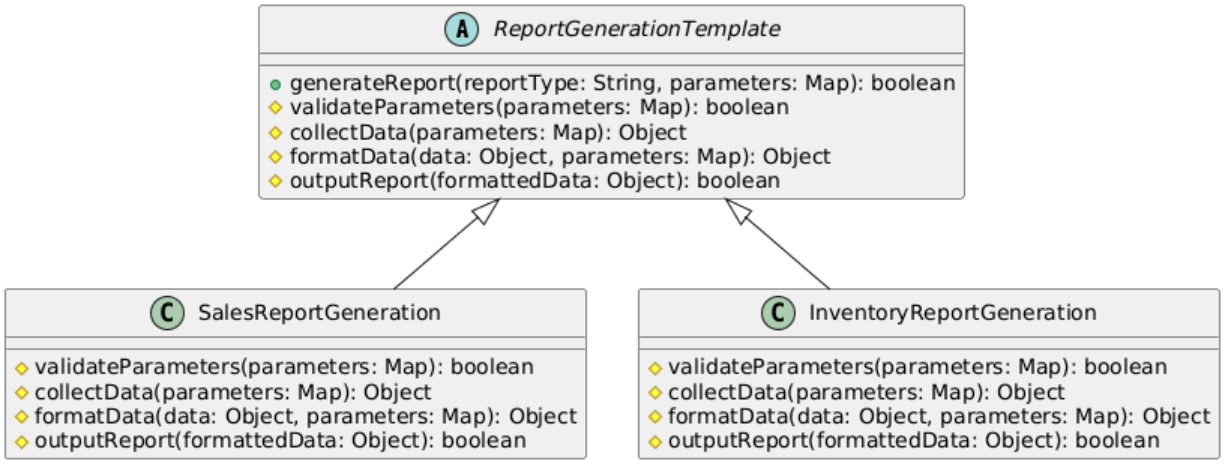
- Duplicate validation/formatting workflow logic
- Hard-to-extend report types

Template Method Solution

Use a template to enforce a standard report-generation pipeline, while allowing subclasses (sales/inventory) to override:

- validation rules (if needed)
- data assembly rules
- output formatting selection points

UML



Justification

Template Method is suitable because:

- It standardizes report generation stages.
- It isolates report-type differences cleanly.
- New report types can be added by introducing a new subclass without rewriting the pipeline.

Implementation Status

Implemented in:

- `ReportGenerationTemplate`
- `SalesReportGeneration`
- `InventoryReportGeneration`

---

3. Mediator Pattern

3.1 Case M1 — Customer Interaction Coordination (Views Orchestration)

Problem

The customer flow uses multiple UI components/views:

- `ProductCatalogView`
- `CheckoutView`
- `PaymentView`
- `OrderTrackingView`

If these components call each other directly, the system becomes:

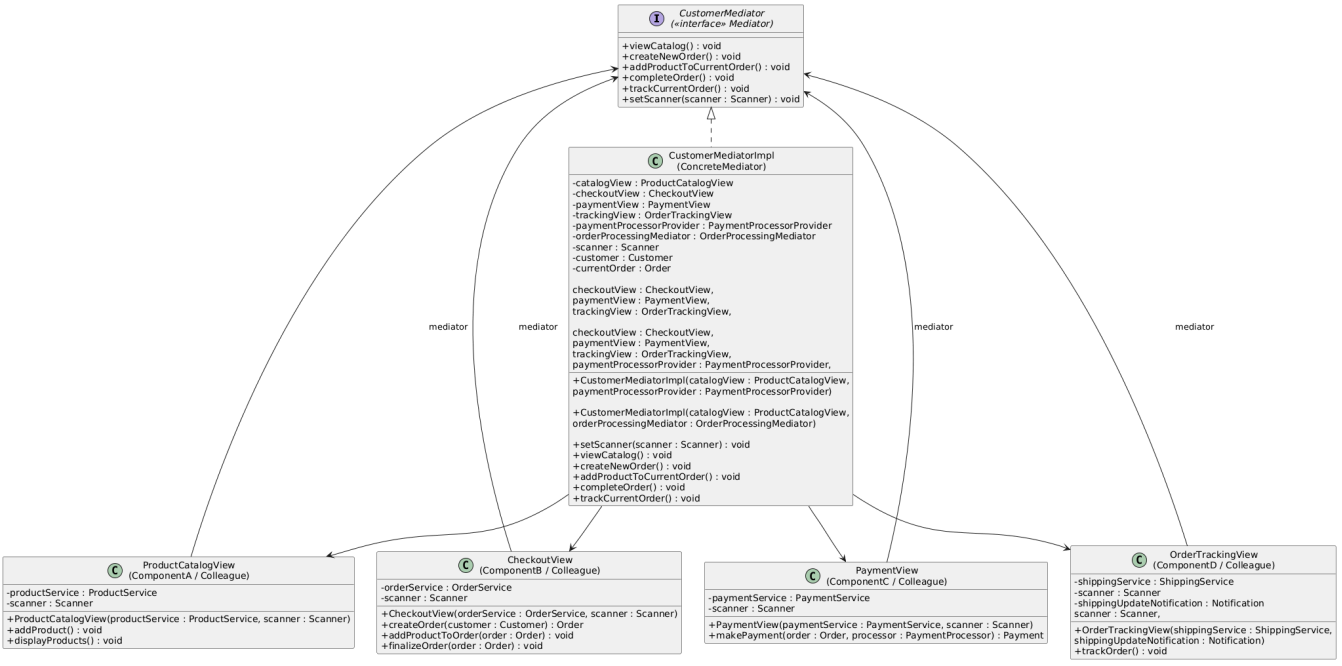
- Tightly coupled (changes ripple across views)
- Difficult to modify interaction flow
- Hard to reason about responsibilities

Mediator Solution

Introduce a mediator that:

- Coordinates the interaction sequence and dependencies
- Centralizes customer flow logic
- Reduces coupling between UI components

UML



Justification

Mediator is appropriate because:

- Many components interact in a complex sequence.
- Central coordination simplifies changes (e.g., adding a new step/view).
- It isolates workflow logic from individual UI components.

Implementation Status

Implemented in:

- `CustomerMediator` (interface)
- `CustomerMediatorImpl` (concrete mediator)

3.2 Case M2 — Order Processing Workflow Coordination (Services + DAOs + Template Method)

Problem

Completing checkout requires coordinating multiple subsystems:

- `OrderService`
- `PaymentService`
- `PaymentInterface` (DAO)
- `ShipmentInterface` (DAO)
- plus selecting the correct Template Method implementation (physical/digital)

Without a mediator, this coordination would:

- Spread across UI/service layers
- Increase coupling and duplicated orchestration code
- Make workflow changes risky

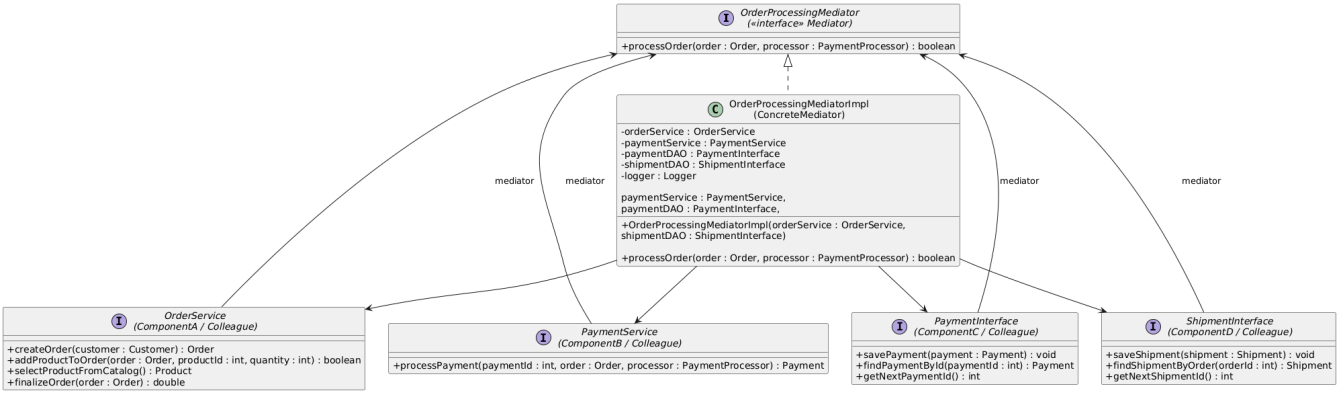
Mediator Solution

Use a dedicated mediator that:

- Receives the order + payment processor
- Determines processing type (physical vs digital)

- Selects and executes the corresponding Template Method workflow
- Ensures centralized error handling and workflow consistency

UML



Justification

Mediator is appropriate because:

- It centralizes and encapsulates workflow orchestration across multiple components.
- It reduces direct dependency chains among services/DAOs.
- It keeps the checkout flow maintainable and extensible.

Implementation Status

Implemented in:

- `OrderProcessingMediator` (interface)
- `OrderProcessingMediatorImpl` (concrete mediator)

---

4. Strategy Pattern

4.1 Case S1 — Payment Execution Strategy (Runtime Payment Behaviour)

Problem

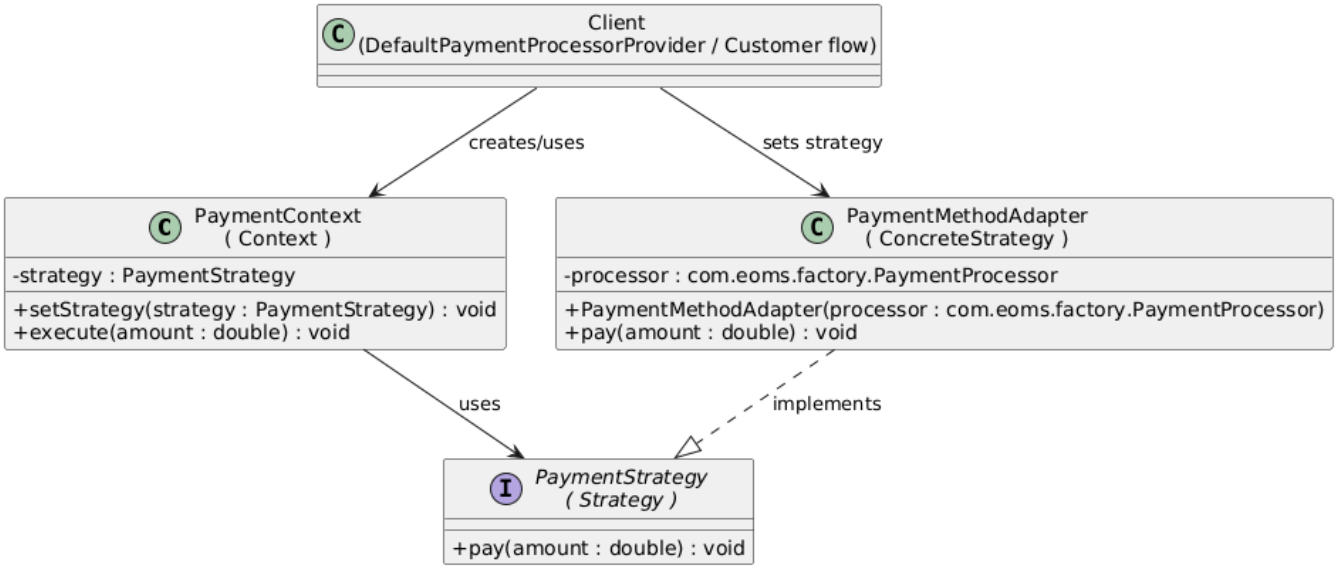
E-OMS supports multiple payment methods and processors. The system must be able to:

- Choose a payment approach at runtime (credit card / PayPal / COD)
- Execute payment without hardcoding conditional logic into the main checkout flow

Strategy Solution

Use a Strategy interface to represent “payment behaviour”, and a Context to execute it.
A concrete strategy adapts the internal payment processor abstraction so the Context stays stable.

UML



Justification

Strategy is appropriate because:

- Payment behaviours are interchangeable at runtime.
- It removes conditional logic from business flow code.
- Adding new payment behaviour becomes additive (new strategy), not modifying existing code.

Implementation Status

Implemented in:

- `PaymentStrategy`
- `PaymentContext`
- `PaymentMethodAdapter`

4.2 Case S2 — Report Formatting Strategy (Interchangeable Export Behaviour)

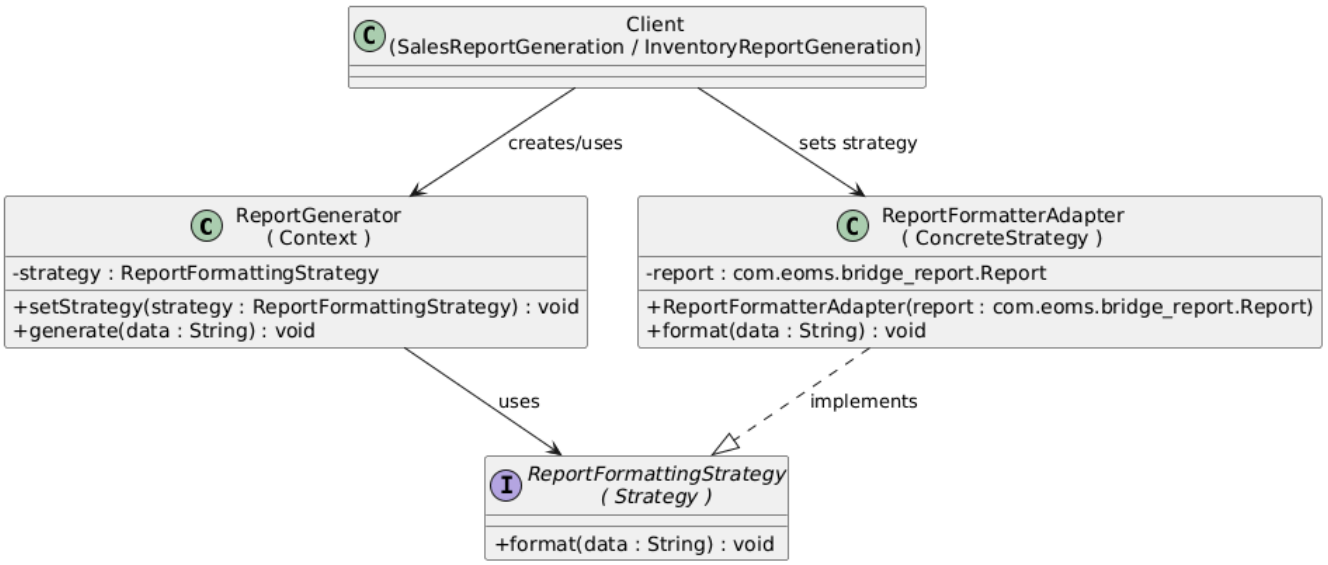
Problem

Reports must be exported using different formatting behaviours (PDF-style vs Excel-style).
The formatting choice should be replaceable without rewriting report generation flow.

Strategy Solution

Use a Strategy interface to represent formatting behaviour and a Context to execute formatting.
A concrete strategy adapts bridge-based report objects into a simple formatting strategy contract.

UML



Justification

Strategy is appropriate because:

- Formatting behaviour varies independently from report generation workflow.
- It enables runtime selection of formatting behaviour.
- It supports extension (new formats) with minimal impact.

Implementation Status

Implemented in:

- `ReportFormattingStrategy`
- `ReportGenerator`
- `ReportFormatterAdapter`

---

5. Observer + Singleton Patterns (Combined Case)

5.1 Case OS1 — Order Events Notification (Central Event Manager + Multiple Listeners)

Problem

When key order lifecycle events occur (created/finalized/paid), multiple independent reactions may be needed:

- Send email notification
- Send SMS notification
- Send WhatsApp notification

If these reactions are hardcoded into services, the system becomes:

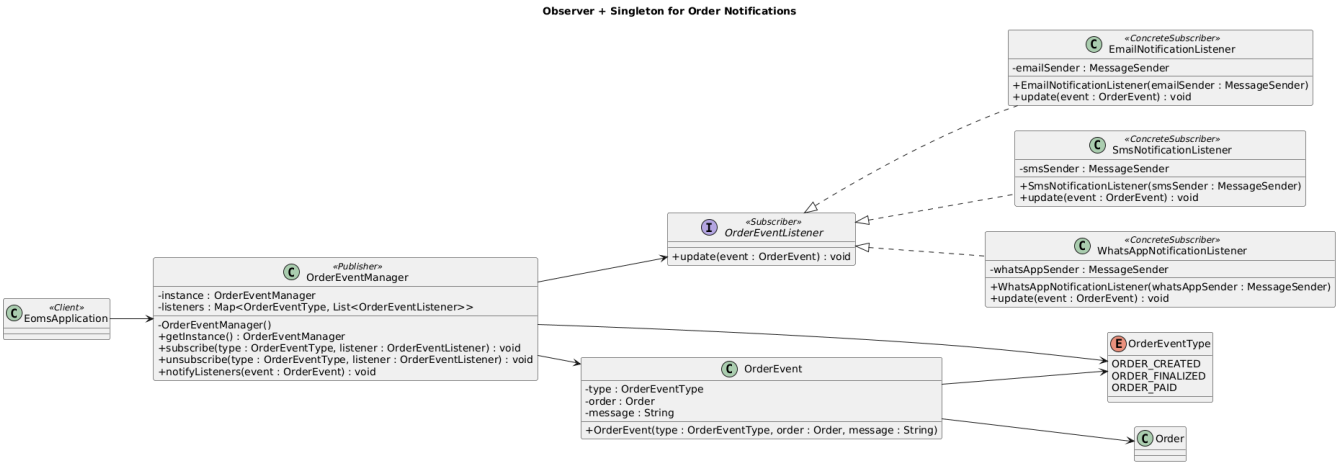
- Hard to extend (new notification channels require modifying services)
- Tightly coupled to concrete notification implementations
- Difficult to enable/disable listeners dynamically

Additionally, the event dispatcher must be **globally consistent** (all parts of the system publish to the same event hub).

Observer + Singleton Solution

- **Observer**: Represent notification handlers as listeners and notify them when order events occur.
- **Singleton**: Implement the event manager as a single global instance so all publishers/listeners share the same dispatcher.

UML



Justification

This combination is appropriate because:

- Observer provides decoupled “publish/subscribe” behaviour for multiple handlers.
- Singleton ensures a single, consistent event dispatcher is used system-wide.
- Adding/removing notification channels becomes plug-in style (new listener class, no changes to publishers).

Implementation Status

Implemented in:

- `OrderEventManager` (**Singleton**)
- `OrderEventListener` + concrete listeners:
  - `EmailNotificationListener`
  - `SmsNotificationListener`
  - `WhatsAppNotificationListener`
- Event types/data:
  - `OrderEventType`
  - `OrderEvent`

---

6. Iterator Pattern

6.1 Case I1 — Traversing Orders Collection Without Exposing Representation

Problem

The system stores orders as a collection (e.g., list). Some features (admin reporting, tracking, or order browsing) may require iterating through orders.

If the internal collection is exposed directly:

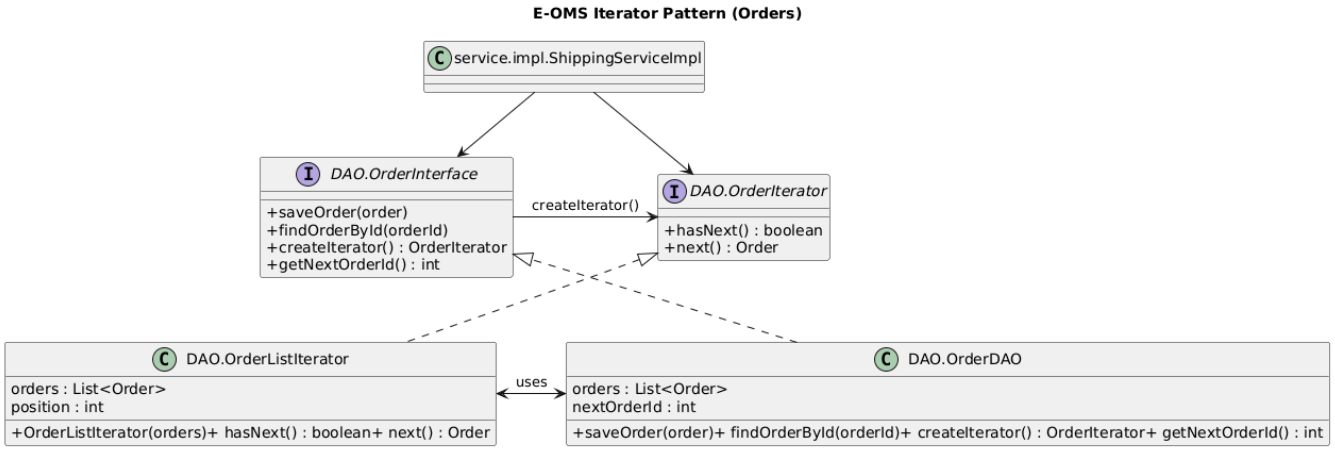
- Clients become dependent on the concrete collection type
- Changing storage representation forces widespread changes

Iterator Solution

Define a custom iterator interface and a concrete iterator implementation that:

- Traverses a list of orders
- Hides underlying collection structure from clients

UML (Placeholder)



Justification

Iterator is appropriate because:

- It provides a stable traversal interface.
- It prevents exposing internal data structures.
- It supports future changes to storage/collection type with minimal impact.

Implementation Status

Implemented in:

- `OrderIterator`
- `OrderListIterator`

---

7. Summary and Conclusion

The selected behavioural patterns solve concrete workflow and extensibility problems in the E-OMS project:

- **Template Method**: Standardizes multi-step workflows while allowing controlled variation (order processing, report generation).
- **Mediator**: Centralizes coordination between multiple components to reduce coupling (customer flow, order processing workflow).
- **Strategy**: Enables interchangeable behaviours at runtime (payment execution, report formatting).
- **Observer + Singleton**: Implements scalable event-driven notifications with a single consistent dispatcher.
- **Iterator**: Supports traversal of collections while hiding internal representation details.

Github link: https://github.com/Kiro-create/NexOrder